

ARC 107 C Shuffle Permutation题解

首先可以发现，若 i 行和 j 行可以交换，则无论怎么交换列 i 行和 j 行还是可以交换的，也就是说行和列是互不影响的，所以我们可以将行与列分开来考虑。则答案等于行的交换方式 $\times$ 列的交换方式。

我们先来考虑列：

一张图上有 n 个点，初始没有一对点之间有边。若 i 列和 j 列可以交换，我们将 i 号点和 j 号点连边。

我们可以发现，一个联通块内的每一对点是可以在不改变其它点的基础上两两交换的。假设我们要交换 i 号点和 j 号点。若存在一条边 $i - a - b - c - d - j$ ，我们可以按照如下方式交换：

$$\begin{array}{c} i - a - b - c - d - j \\ a - i - b - c - d - j \\ \dots \\ a - b - c - d - i - j \\ a - b - c - d - j - i \\ a - b - c - j - d - i \\ \dots \\ j - a - b - c - d - i \end{array}$$

有了这个性质也就可以说明，一个联通快内的所有边可以组成这些边的所有的排列。

则列的交换方式为 $\prod size_i!$ ， $size_i$ 表示 i 号联通块的大小。

对于行也是类似，只需要进行和上面一样的操作就可以了。

Code:

[kawatea](#):

这个代码并不像我所说的进行两次操作分别来求行的方案数和列的方案数。而是建在了一张图里，行的编号为1到 n ，列的编号为 $n + 1$ 到 $2 \times n$ ，这样就可以保证行和列互不影响，且一遍就可以算出答案。

```
#include <cstdio>

using namespace std;

const int mod = 998244353;
int a[50][50];
int par[100];
int cnt[100];
long long fact[51];

int find(int x) {
    if (par[x] == x) return x;
    return par[x] = find(par[x]);
}

void unite(int x, int y) {
    x = find(x);
    y = find(y);

    if (x == y) return;

    par[x] = y;
```

```

}

int main() {
    int n, k, i, j, l;
    long long ans = 1;

    scanf("%d %d", &n, &k);

    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            scanf("%d", &a[i][j]);
        }
    }

    for (i = 0; i < n * 2; i++) par[i] = i;

    for (i = 0; i < n; i++) {
        for (j = i + 1; j < n; j++) {
            for (l = 0; l < n; l++) {
                if (a[i][l] + a[j][l] > k) break;
            }

            if (l == n) unite(i, j);

            for (l = 0; l < n; l++) {
                if (a[l][i] + a[l][j] > k) break;
            }

            if (l == n) unite(n + i, n + j);
        }
    }

    for (i = 0; i < n * 2; i++) cnt[find(i)]++;

    fact[0] = 1;
    for (i = 1; i <= n; i++) fact[i] = fact[i - 1] * i % mod;

    for (i = 0; i < n * 2; i++) ans = ans * fact[cnt[i]] % mod;

    printf("%lld\n", ans);

    return 0;
}

```